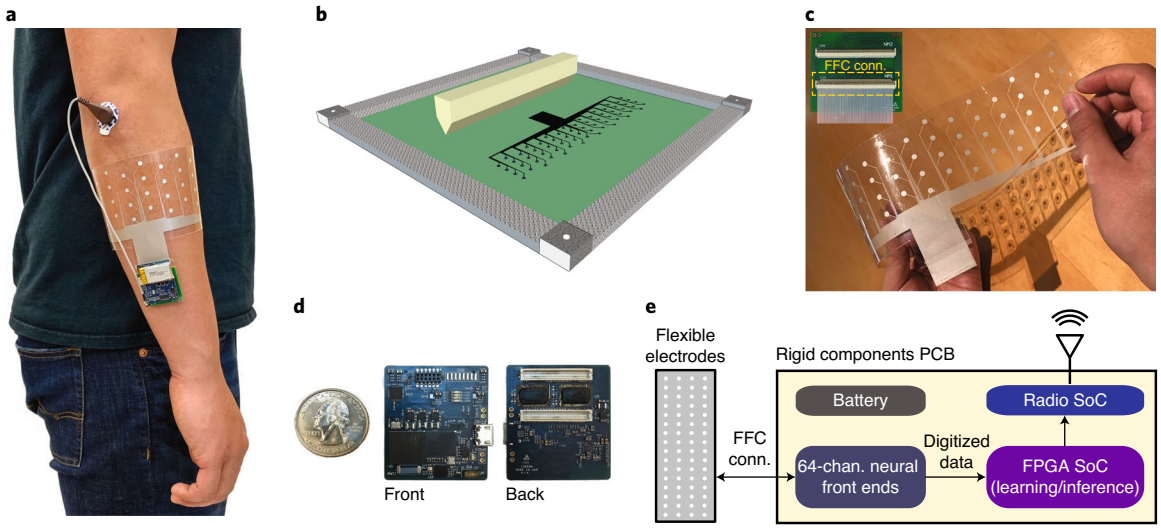


Fig. 1 | Wearable biosensing system for sEMG. a, The device on the forearm of a participant. **b**, Illustration of the screen-printing process. A squeegee and a screen are used to print sEMG electrode arrays onto flexible substrates. The electrode pattern is defined by features on the screen. **c**, The custom-designed, flexible 16 × 4 array of electrodes that conforms to the forearm to provide high-density, large-area sEMG recordings without individual wires. An adapter PCB with a flat flexible connector (FFC conn.) is used to interface the flexible electrode array with the rigid components. **d**, The miniaturized, eight-layer PCB that accommodates the complex, rigid components responsible for sensing, processing and telemetry. **e**, Block diagram of the main components constituting the wearable system.

for physiological signals such as sEMG[32], electroencephalography (EEG)[33] and electrocorticography (ECoG)[34], discriminating up to five classes on offline datasets.

To build our system, we used a hybrid method for interfacing soft conformal sensors and hard silicon-based integrated circuits (ICs). In particular, we combined a sensing electrode array printed on a flexible substrate (Fig. 1b,c) with a miniaturized printed circuit board (PCB) that includes complex sensing, processing and telemetry components (Fig. 1d,e). The large-area, high-density electrode array has 64 channels and was fabricated by screen printing using conductive and dielectric inks on a thin polyethylene terephthalate (PET) sheet. The PET sheet is highly flexible and conforms well to the complex three-dimensional (3D) form of forearm muscles during contractions and relaxations. The system uses a low-power custom application-specific integrated circuit (ASIC) to collect the sEMG data for classification. The custom-designed ASIC[35] can record and digitize the sEMG signals from multiple channels close to the electrode sites, reducing the need for bulky cable connections between electrodes and signal conditioning circuitry. We validated our biosensing device through a series of real-time experiments that model signal variations from real-world use cases. We obtained a hand gesture classification accuracy of 97.12% when classifying 13 hand gestures using only a 4 s window of training data per gesture. We demonstrated that the classification model can be updated locally to classify a larger set of hand gestures, as well as to recover from accuracy degradation due to theparticipant moving to a new arm position, replacing the device or wearing the device for a prolonged period of time.

Hybrid flexible biosensor design and fabrication

To create the flexible skin-worn interface, we screen-printed a uniform 4 × 16 array of circular electrodes (diameter r, 4.3 mm) and connective traces on a flexible PET substrate using conductive silver ink (NovaCentrix FG57b) sintered by photonic curing (Fig. 1b,c)[36]. The silver ink formulation included binders to help with cohesion of the silver microflakes as well as adhesion to the substrate. Compared to high-density electrode array fabrication methods reported in the literature[28,32,37,38], our screen-printing solution with photonic sintering allows the use of temperature-sensitive substrate materials and requires only subsecond curing time for cheaper and faster large-scale production. A dielectric encapsulation layer (NovaCentrix DE-SP1) was subsequently printed with via holes for exposing the electrode pads while insulating the conductive traces from the skin. Each printed layer was 15 µm thick. Overall dimensions (29.3 cm × 8.2 cm) were chosen to wrap around the entire circumference of an above-average-sized forearm, capturing activity of the extrinsic flexor and extensor muscles involved in finger movements with low inter-electrode pitch in both the proximal–distal and medial–lateral directions. Four electrodes per column were spaced 14.3 mm apart, and the columns were spaced 17.8 mm apart with vertical offsets of 7.15 mm.

The flexible electrode array was interfaced to a custom, eight-layer PCB (Fig. 1d,e) using a flat flexible connector (FFC) on a two-layer adapter board (Fig. 1c). All 64 channels from the electrode array were sampled and digitized at 1 kS s^{-1} using a custom neural interface IC[35] with very low power consumption (700 µW) in a small form factor. sEMG signals were processed within a system-on-a-chip field-programmable gate array (SoC FPGA), where we implemented a full training and classification algorithm to make the system entirely standalone. The device connected wirelessly to a base station computer using a 2.4 GHz radio SoC for reconfiguration and data logging, if needed. The board was powered using a single 3.7 V, 240 mAh lithium-ion battery weighing 4 g. The total weight of the wearable system was 26 g, with the computational unit PCB weighing 6 g and the electrode array and adapter weighing 16 g. In this configuration, battery life during continuous gesture recognition was ~6 h, with power dissipation being dominated by raw data packetization and wireless transmission at greater than 1 Mbps (Supplementary Fig. 7).

On-body characterization of the electromyography acquisition

Using our device, we collected and analysed an offline sEMG dataset from five participants across multiple wear conditions simulating

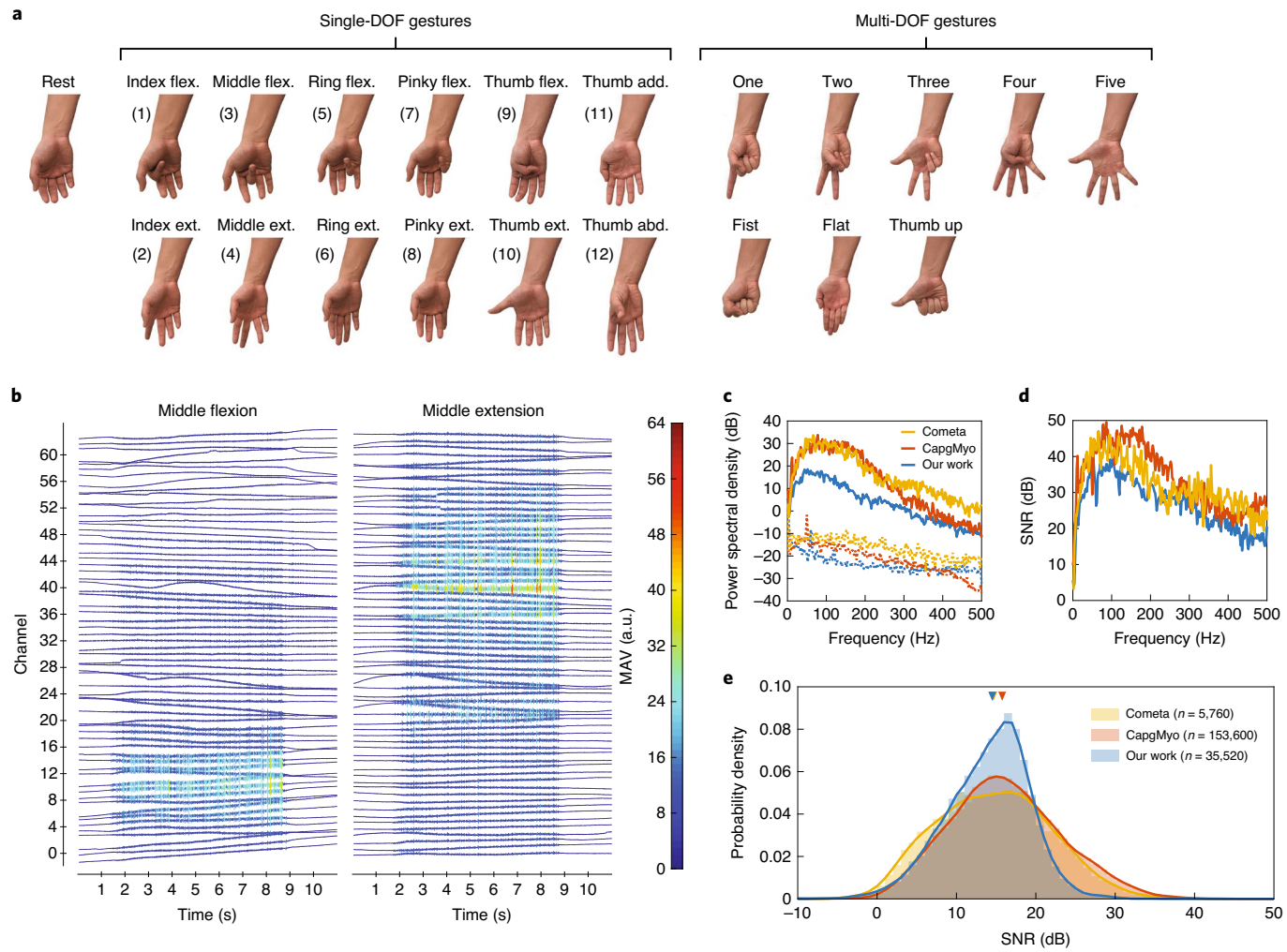


Fig. 2 | Hand gesture classes used in the study and sEMG recording characteristics. a, The single-DOF gesture subset includes individual finger flexions (flex.) and extensions (ext.) along with the 'rest' gesture. The multi-DOF gesture subset includes common, isometric hand postures involving multiple fingers. add., adduction; abd., abduction. **b**, Example raw waveforms recorded from all 64 channels during middle finger flexion and extension. Channels are organized starting with channel 0 on the surface nearest the radius, with increasing channels wrapping around the anterior side of the forearm to the surface nearest the ulna (~channel 32), and then around the posterior side back to the radius. Each 11 s gesture trial is divided into 1.5 s rest, a 2 s transition period to the gesture, a 4 s hold period, a 2 s transition period back to rest and 1.5 s rest based on the instructions given to the participant. The colour of the waveform indicates the local amplitude of the sEMG, as measured by MAV calculated over 50 ms segments. **c**, Example Welch's power spectral density estimates of sEMG recordings from single channels of three systems, including a commercial sEMG interface with individually placed bipolar electrodes (Cometa + Dormo, NinaPro DB4[39–41]), a benchtop high-density sEMG acquisition set-up (CapgMyo[42]) and our system. Solid lines represent the signal spectrum during performance of the middle extension gesture, while dotted lines represent the signal spectrum during rest. Spectral densities are normalized to the 0 Hz component of the rest spectrum for each recording system, for ease of comparison. **d**, Spot SNR for the example performances in **c**, measured as the ratio between the middle flexion spectrum and the rest spectrum. **e**, Distribution of overall SNR for the three compared systems for all channels during the performance of all single-DOF gestures. Shaded areas are the probability density histograms of SNR values. Lines are a Gaussian kernel fit to the distribution. Downward pointing triangles represent the medians.

typical use cases (Fig. 2 and Supplementary Note 1). The dataset included flexion and extension of different finger degrees of freedom (DOF; Fig. 2a), with single-degree-of-freedom (single-DOF) gestures being performed in four different situational contexts and multiple-degree-of-freedom (multi-DOF) gestures being performed in only the baseline situational context (Supplementary Note 1). The device was worn by each participant as shown in Fig. 1a, with the centre of the array roughly aligned to the ulna of their dominant arm (Methods). A single drop of conductive gel was placed on each electrode to improve skin contact and adhesion. Signals were recorded in a single-ended configuration, with an ECG Ag/AgCl electrode on the elbow used as reference. Figure 2b shows example waveforms recorded from all 64 channels during the flexion and extension of the middle finger DOF. Each 50 ms segment of each waveform is coloured based on the mean absolute value (MAV) feature derived from that segment, indicating the local amplitude of the sEMG. Activity in contrasting subsets of channels (overlaying the anterior flexor muscles or posterior extensor muscles) can be seen in the antagonistic movements.

Signal quality, as acquired using our wearable biosensing system, was compared to available recordings of the same gestures using a traditional commercial sEMG interface (Cometa used in NinaPro DB4)[39–41] as well as a benchtop high-density sEMG acquisition set-up (CapgMyo)[42]. To compare the power spectrum of the recordings, we computed the Welch's power spectral density estimate for the channel with the highest signal-to-noise ratio (SNR) while performing

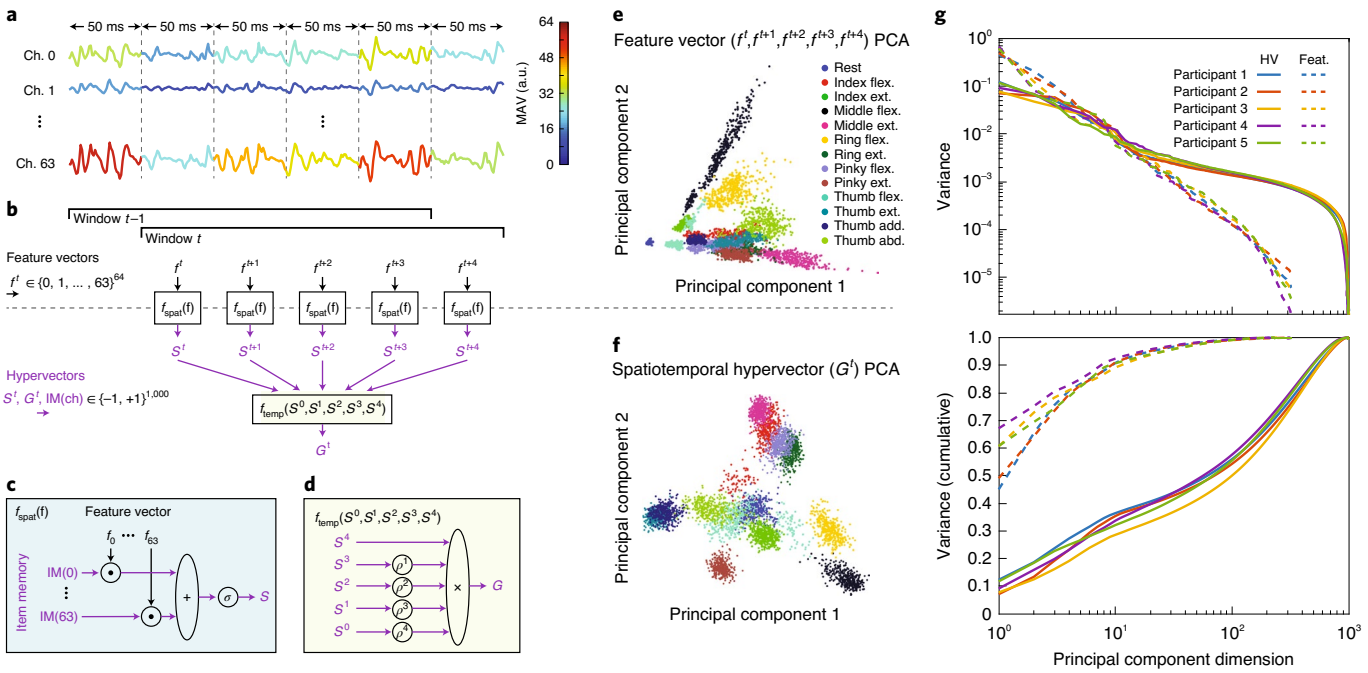


Fig. 3 | Hyperdimensional computing algorithm for projecting windows of sEMG data into hypervectors. a, Example sEMG data and MAV feature values. The three example traces shown are divided into 50 ms feature segments, with segments colour-coded based on MAV feature values. **b**, A single classification window at time t consists of five feature segments totalling 250 ms, and consecutive classification windows overlap by 80%. Each feature segment f^t is encoded into a spatial hypervector $S^t = f_{spat}(f^t)$. The five consecutive spatial hypervectors are then bound temporally to form a spatiotemporal hypervector representing the entire window $G^t = f_{temp}(S^t, S^{t+1}, S^{t+2}, S^{t+3}, S^{t+4})$. The dashed grey line shows the division between the low-dimensional feature domain and the high-dimensional hypervector domain. **c**, Spatial encoding function f_{spat} consists of computing a weighted sum of immutable item memory (IM) hypervectors IM(ch) weighted (·) by their corresponding features f_{ch}. The sum is bipolarized (σ) back to ±1. **d**, A temporal encoding function binds consecutive spatial hypervectors S^t together through permutation by k bits (ρ^k) and element-wise multiplication (×) operations. **e**,**f**, Principal component analysis for all classification windows from five trials of 13 single-DOF gestures. Analysis is shown for one participant in a baseline context. The top two principal components are plotted for features (**e**) and hypervectors (**f**) for each window. **g**, Proportion of explained variance for all principal component dimensions per dimension (top) and cumulative (bottom). Solid lines represent dimensions of the hypervector (HV) set; dashed lines represent dimensions of the feature (Feat.) set.

the middle extension gesture (gesture 4), the gesture that produced the highest SNR across all three devices (Fig. 2c). The frequency contents of the different recordings were qualitatively similar. Spot SNR was calculated by comparing the power spectrum during a gesture performance to the power spectrum during rest (Fig. 2d). Overall SNR (calculated as the integral of spot SNR) varied for different channels and different gestures (Fig. 2e). The Cometa and CapgMyo systems exhibited better peak SNR for the best channels and associated best gestures, probably because those systems consist of differential, bipolar recording configurations with improved common-mode noise rejection. Despite this, median SNRs for the three systems were approximately the same. Furthermore, SNR values for our system were similar before and after long, 2 h wear sessions, during which participants could sweat and the conductive gel could smear (Supplementary Fig. 2).

Gesture recognition algorithm with adaptive learning

We implemented an HD computing algorithm[31] for training and inference of hand gestures. Raw data or preprocessed features (which are continuous-valued and low-dimensional) are projected onto 1,000D bipolar ($\{-1,+1\}^{1,000}$, for simplicity in describing mathematical operations) or binary ($\{0,1\}^{1,000}$, for implementation with digital logic) hypervectors with information fully distributed across all bits, analogous to the way the human brain utilizes vast circuits of neurons and synapses for learning and recall. Learning for classification tasks can be implemented with low-complexity element-wise majority functions for computing cluster centroids, or prototype hypervectors, which each represent a class. Similarly, inference can be implemented as a nearest neighbour search within the learned class prototype hypervectors to a given query hypervector using Hamming distance.

Figure 3 describes the algorithm for projecting sEMG data into hypervectors. Importantly, the same projection process is used for both learning and inference, allowing reuse of hardware modules for both in-sensor training and classification[43]. Signals are processed in 250 ms sliding windows with 80% overlap, resulting in a latency of 250 ms and classification throughput of 20 predictions per second, which are suitable for real-time control[44,45] (Fig. 3a). Each window is subdivided into five 50 ms feature segments, with the MAV feature being calculated per channel for each segment. Feature selection and hyperparameter tuning are described in more detail in Supplementary Note 2.

For each segment index t, the feature vector f^t is projected into a hypervector S^t representing spatial information, that is, which electrode channels have higher feature values (Fig. 3b,c). Electrode channels are represented by unique, pseudo-random bipolar hypervectors stored in an item memory (IM), which remains unchanged throughout the training and deployment of the algorithm. Each electrode channel vector IM(ch) is scaled by its corresponding feature value f^t_{ch}, and all scaled vectors are summed element-wise. The resultant spatial hypervector is then formed as $S^t = \sigma\left(\sum_{ch} \mathrm{IM(ch)} f^t_{ch}\right)$, where σ is a bipolar threshold function that turns positive elements to +1 and negative elements to −1 (Fig. 3c).

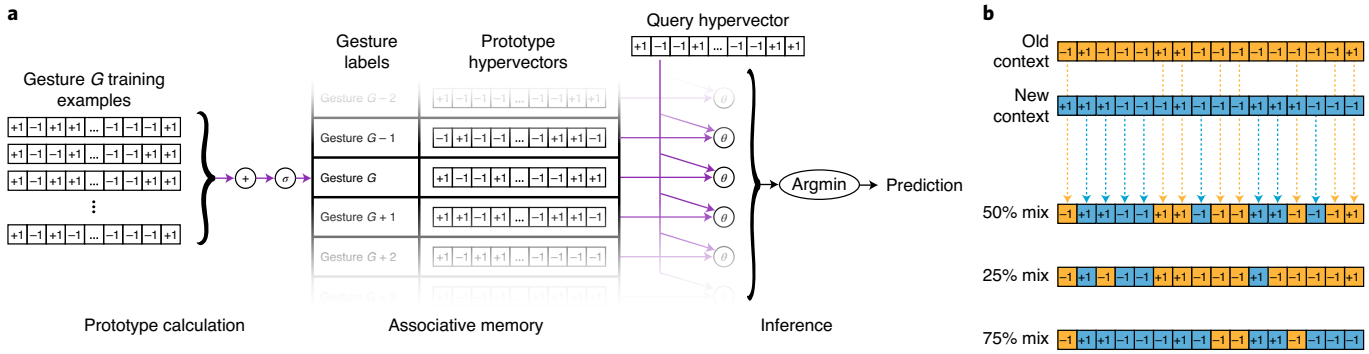


Fig. 4 | AM operations for training, accessing and contextual updating. a, Usage of the AM, from prototype calculation and storage for training (left) to Hamming distance calculation for inference (right). Prototype hypervectors are created from training examples for each gesture by taking the majority of each element across all examples. For bipolar hypervectors, this can be implemented as a sum (+) and bipolarization (σ). During classification of a query vector, the nearest neighbour among stored prototypes is found using the Hamming distance (θ) as the metric. **b**, Bitwise merge of two prototype hypervectors approximates finding weighted centroids of clusters from different contexts of the same gesture class. The updated hypervector randomly takes elements from the stored prototype hypervector and the new context prototype hypervector. The proportion of bits taken from each hypervector determines the relative weight of each context in the updated prototype.

The group of five spatial hypervectors is then encoded into a single spatiotemporal hypervector G^t (Fig. 3b,d). The order in which they occur is encoded by performing a bitwise rotation by k bits (ρ^k), with increasing k for older hypervectors (Fig. 3d). The rotated hypervectors are multiplied together element-wise, resulting in a single 1,000D bipolar hypervector representing the 250 ms window.

To visualize this projection into HD space, we performed principal component analysis (PCA) on the classification windows consisting of five feature vectors, as well as on projected spatiotemporal hypervectors from trials of the single-DOF gestures performed by one participant (Fig. 3e–g). The top two principal components (Fig. 3e,f) show general clustering of different gestures, with hypervector clusters having more circular groupings that can be better represented by their cluster centroids. The dimensionality of the overall dataset can be seen in the proportions of variance explained by each principal component dimension (Fig. 3g). For the hypervector dataset, nearly 500 principal component dimensions are required to explain 90% of the variance, whereas for feature vectors only ~10 dimensions are needed. This indicates that information is properly distributed across the large number of elements.

The encoded spatiotemporal hypervectors can be used either as training examples for creating or updating a model or as queries for inference using a trained model (Fig. 4). A prototype hypervector for each class is formed by computing the class centroid. For binary and bipolar hypervectors, this amounts to finding the majority of each element across all training examples. These prototype hypervectors are then stored in an associative memory (AM), an entirely feedforward operation with a single pass over training data (Fig. 4a). This is in contrast to other neuro-inspired approaches in which training often employs sophisticated, iterative frameworks and is much more computationally demanding than classification (for example, gradient descent with backpropagation[46]). Adding new classes to the model simply involves adding new prototype hypervectors to the AM, again differentiating HD computing from other algorithms that may require full retraining or modifications to the architecture. Once all prototypes have been computed and stored, classification involves finding the nearest-neighbouring prototype to a query hypervector (Fig. 4a).

To update a stored prototype to incorporate information about a new situational context for the same gesture, an approximated majority vote can be performed by merging elements from both the stored and the newly computed prototype hypervectors (Fig. 4b). Different proportions of new and old elements can be taken to weigh the contribution of each context. As more contexts are encountered, this proportion can be changed to tune the decay rate to avoid catastrophic forgetting of initial contexts. Very little additional overhead is required to perform model updates, compared to standard gesture recognition algorithms like support vector machines, in which new support vectors must be selected among new and old training examples[47,48], or linear discriminant analysis, in which training set statistics must be recalculated to a relatively high degree of precision[49,50].

Real-time in-sensor implementation of adaptive learning and classification

We tuned model hyperparameters and validated the HD algorithm using the offline dataset (Supplementary Note 2) before optimizing it for efficient implementation using hardware description language (HDL) for synthesis on the device's FPGA. Raw 15-bit analogue-to-digital converter codes were used as the input for feature extraction, with incremental calculations of the 50-sample MAV performed with each new sample. The implemented arithmetic operations consisted only of addition, two's complement inversion, and arithmetic right shift for division. Features were quantized and saturated to 6-bit integers based on analysis of the offline dataset, optimizing for the dynamic range (range divided by step size) given the arithmetic requirements.

The 1,000-dimensional item memory elements were generated sequentially using a cellular automaton with a hardcoded seed[51] for a smaller memory footprint. Subsequent processing steps were exactly as described in the previous section on HD classification architecture, replacing algebraic operations on bipolar hypervectors with Boolean operations on binary vectors[52]. A shift register consisting of 21 hypervectors was used as the AM to store trained prototype hypervectors and search for the closest class during inference. A single contextual update with 50% weighting was enabled for each AM entry by merging a predetermined set of 500 bits from a newly trained prototype into the stored one.

The implementation utilized 84% of resources available on our FPGA (Supplementary Note 3). Simulations verified that the algorithm (after acquiring the last sample of the last 50 ms feature window) had a latency of 539 cycles, or 26 µs when running at 20.48 MHz (Supplementary Note 4). The FPGA modules for

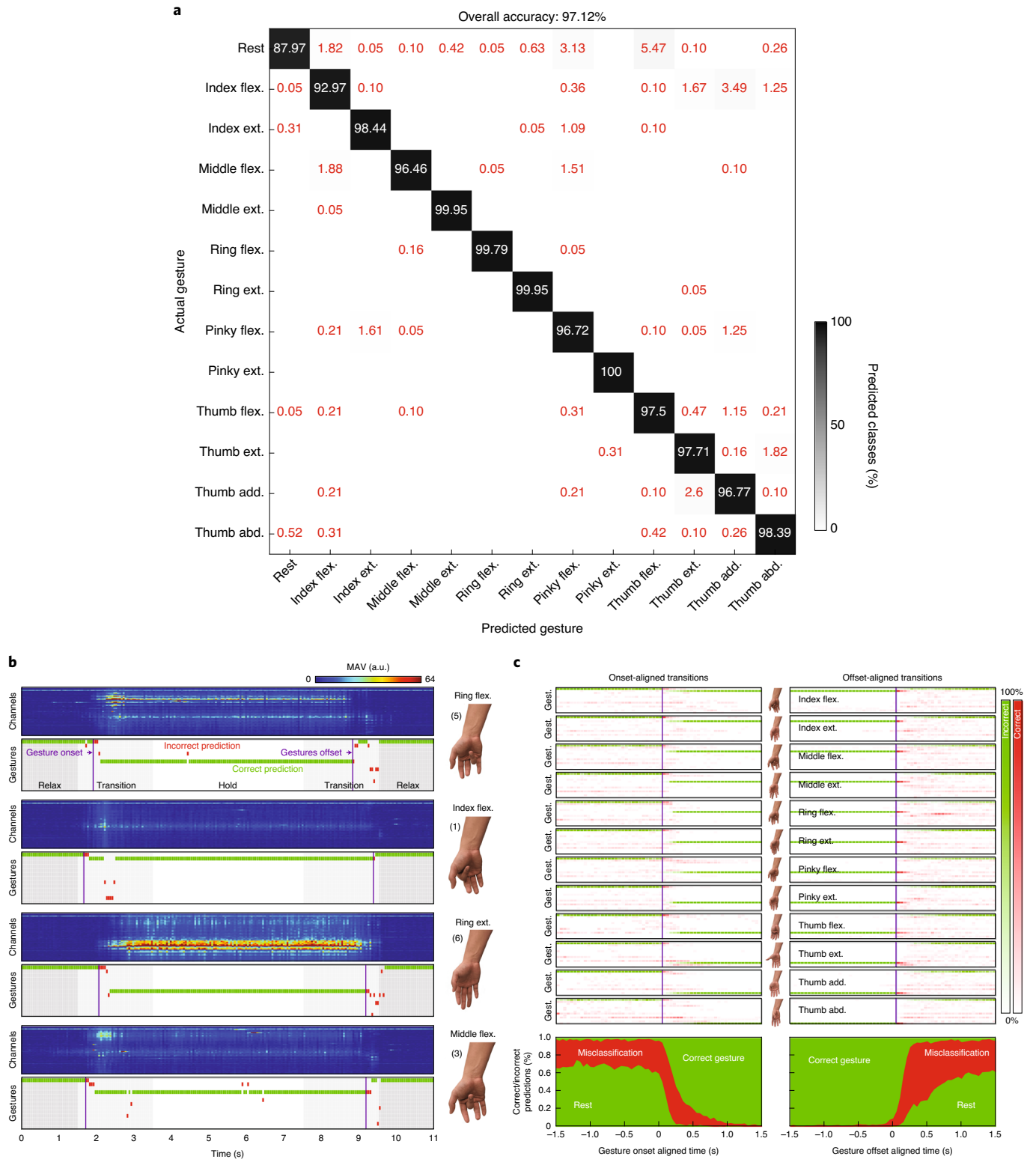


Fig. 5 | Real-time, in-sensor classification performance in the baseline context. a, Classification confusion matrix during 4 s hold periods of baseline context testing for real-time experiments. White text values are percentages of correct predictions, red text value are percentages of incorrect predictions. The greyscale coloured background represents the proportion of predicted classes. **b**, Examples of real-time prediction outputs for four gesture trials. For each gesture trial, the top plot displays the features calculated over 50 ms segments for all channels. The bottom plot shows the 20 Hz classification results relative to trial timing. Purple vertical lines denote offline-calculated gesture onset in the first transition period and gesture offset in the second transition period. The vertical position of predictions represents gesture class, and predictions are colour-coded based on accuracy with respect to offline-estimated ground truth labels. **c**, Gesture onset-aligned (left) and offset-aligned (right) prediction statistics over all baseline context tests. Each of the first 13 rows shows the averaged 20 Hz classification results for a given gesture, with more darkly coloured blocks indicating a higher proportion. Green blocks indicate correct predictions; red blocks indicate incorrect predictions. The bottom plots show the percentages of correct (green) and incorrect (red) classifications over time, relative to the onset and offset times.

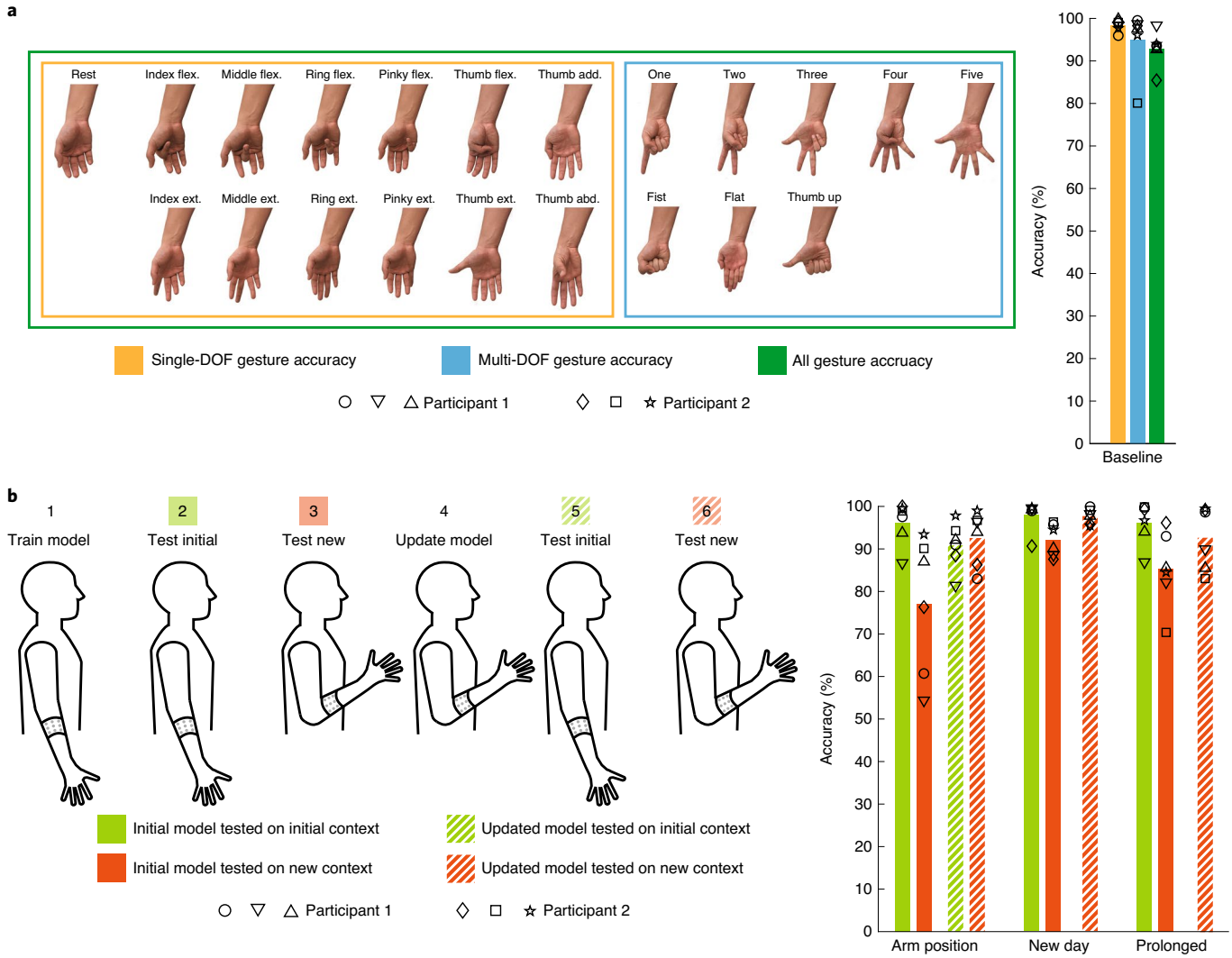


Fig. 6 | In-sensor training, update and classification results. Two participants performed each experiment three times. Bars represent the mean accuracy across all six trials, with overlaid data points for each individual trial. **a**, The ability to update the HD classification model with new gestures. An initial model was trained and tested on single-DOF gestures only (yellow bar). The model was then updated with multi-DOF gestures to cover all 21 gestures (green bar). Results for a separate model trained and tested on multi-DOF gestures only are also illustrated (blue bar). **b**, The ability to update the HD classification model with new contexts using only the single-DOF gestures. For three different context changes (arm position, new wear session and prolonged wear), an initial model was trained on the initial context (step 1) and tested on both the initial (step 2) and the new (step 3) contexts (solid bars). The model was then updated using a single trial of each gesture in the new context (step 4), and again tested (steps 5 and 6) on both contexts (striped bars). Updated models were tested only in the new context for the new wear session and prolonged wear experiments because the old context was no longer available.

classification were thus clock-gated to only be active during the first 26 µs after receiving a new sample, consuming no dynamic energy during the remaining time. Simulations and source meter measurements showed that the in-sensor HD algorithm operates with 2.437 µJ per sEMG sample and 4.39 µJ per classification every 50 samples (Supplementary Note 5). Model training and updating consumed similar energy to classification steps. In all, the on-board algorithm accounted for less than 6% of the overall device power.

We verified the online, standalone operation of our device with two participants who were familiar with myocontrol. Four types of experiment were each conducted three times to test varying device usage conditions with all training, inference and updates performed completely on board and in real time. A custom-made graphical user interface (GUI) was used to instruct the user on what gesture to perform, transmit the correct gesture label and the operation mode (train/infer/update) to the device, and log data. Both raw sEMG data and the classified gesture class were streamed back to the GUI and displayed in real time, giving the participant visual feedback during inference modes.

Figure 5 shows real-time classification accuracy and prediction time series from the device during the performance of single-DOF gestures in the baseline context immediately after device donning and with a relaxed arm. Each gesture was performed once during training, including a trial at the beginning for the rest gesture. Participants were instructed to begin each gesture within a 2 s transition period, hold the gesture for a 4 s steady-state period, and relax back to the rest gesture within a second transition period, with 3 s of relaxation between each gesture (Fig. 5b). The GUI was configured to only enable the training mode on the device during the 4 s steady-state period. Exactly the same procedure was then repeated with the device in inference mode. Figure 5a shows the classification accuracy and confusion matrix for the 4 s steady-state periods of each gesture.

Table 1 | Comparison of this work with other sEMG systems

	Amma et al. (2015)[26]	Liu et al. (2017)[12]	Moin et al. (2018)[32]	Pancholi and Joshi (2019)[29]	Benatti et al. (2019)[30]	Cerone et al. (2019)[28]	This work
Wearable form factor	✗	✓	✓	✓	✗	✓	✓
Electrode type	Flexible PCB	Individual surface patches	Flexible PCB	Ag-AgCl electrode band	Individual Ag-AgCl	Flexible PCB	Flexible screen-printed
Electrodes per array	192	4	64	8	8	32	64
Wireless streaming	✗	✓	✓	✗	✓	✓	✓
In-sensor classification	✗	✓	✗	✓	✓	✗	✓
In-sensor model training	✗	✗	✗	✓	✓	✗	✓
In-sensor adaptive update	✗	✗	✗	✗	✗	✗	✓
Number of classes	27	10	5	6	11	-	21

Actual gesture onset and offset times were determined offline with a combination of visual and changepoint analysis methods so as to estimate a ground truth label (Fig. 5b). Although the classification model was not trained on data that included gesture transitions or relaxation between trials, the device still performed inference during these periods. Gesture onset- and offset-aligned predictions across baseline context tests in all 24 experiments are shown in Fig. 5c. A higher error rate during relax periods (during which the prediction should be the rest gesture) is seen both before and after gesture performance, suggesting that the short periods between gesture movements may be included as training data to improve performance. Classification of the rest state is inherently more difficult, given that our hypervector projection algorithm encodes relative feature values rather than absolute feature values. Additionally, gesture transitions produced errors, requiring about 500 ms from gesture onset (including algorithm latency) to achieve 80% classification accuracy. The larger amount of misclassification following gesture offset is in part due to unintended movements in the opposite direction or of other finger degrees of freedom during transitions back to rest.

Figure 6 shows the steady-state gesture classification accuracy for the different experiments requiring online, in-sensor training and model updates. The first experiment (Fig. 6a) demonstrated the ability to update the HD classification model with new gestures. We trained and tested an initial model with only the single-DOF gesture subset (Fig. 6a, yellow bar) and achieved an average classification accuracy of 98.34%. We then updated the model with the multi-DOF gesture subset and tested on all 21 gestures (Fig. 6a, green bar) with 92.87% accuracy—a 5.47% degradation. Training and testing on the multi-DOF gesture only resulted in a similar accuracy of 95.04% (Fig. 6a, blue bar).

In the next three experiments, we demonstrated the ability to update the HD classification model with new contexts using only the single-DOF gestures. The context changes we explored were new arm position (going from the relaxed position with the arm at the side to the elbow rested on an armrest in an arm-wrestling position), new day with the device doffed and re-donned in between (>16 h apart, mean 19 h), and prolonged wear with the device worn while going about daily activities for 2 h. An initial model was trained and tested in the baseline context (Fig. 6b, solid green bar). The model was then tested in the new context, prior to updating (Fig. 6b, solid red bar). This resulted in, on average, an 11.89% accuracy degradation. The model was then updated in the new context and tested again (Fig. 6b, striped red bar). In the case of arm position, the updated model was also tested in the initial context (Fig. 6b, striped green bar). In each case, accuracy was recovered after updating the model for the new context, with an average improvement of 9.50% compared to the initial model.

A comparison of our device to existing state-of-the-art sEMG systems is presented in Table 1. Our system is the only one tested in an incremental learning framework with in-sensor training, inference in multiple new situational contexts, and incremental updates to adapt to new contexts and recover accuracy. Compared to the systems that are capable of in-sensor classification, ours has the most electrode channels by at least a factor of eight. The custom-designed flexible electrode array and low-power integrated circuit front ends allow us to maintain a small form factor and low power consumption, comparable to systems with a smaller number of channels, which are important for device wearability. A more detailed comparison of device implementations is provided in Supplementary Table 5. Our custom hardware implementation of the algorithm on an FPGA achieved the best classification energy efficiency, even with more gesture classes. Note that the total energy required for the processing (feature extraction and classification) was slightly higher compared to other reports because of the larger number of channels.

Conclusions

We have reported a self-contained, wearable sEMG biosensing system that uses HD computing to process and classify hand gestures. All the functionalities of our system, including data acquisition using a printed flexible electrode interface, as well as training and classification with a machine-learning model, are incorporated into a compact device that requires no external computation. Our biosensing system is comfortable to wear and offers fast initial training as well as on-the-fly adaptation, which are crucial for wearable human–machine interface applications in which physiological signals vary from user to user and are not stationary. In contrast to other state-of-the-art gesture-recognition systems, our device can perform training, inference and model updates locally and in real time to adapt to changing situational contexts.

Our results are promising for the development of standalone wearable devices with full, in-sensor machine-learning capabilities. Two natural extensions of this work are the incorporation of additional situational contexts (such as a larger variety of arm positions) and the incorporation of gesture transitions, which can either be treated as new contexts or as separate classes. As more information is encoded into gesture prototype hypervectors, it may eventually become beneficial to trade off memory footprint for improved classification performance and to represent gesture classes with more than one prototype hypervector. Finally, although our electrode array geometry was designed for forearm sEMG acquisition, the

Table of Contents

Supplementary Note 1. Offline dataset collection 2

Supplementary Note 2. Feature selection and model validation 3

Supplementary Note 3. FPGA resource utilization 9

Supplementary Note 4. Algorithm latency 9

Supplementary Note 5. Power breakdown 10

References 12

Table of Figures

Supplementary Fig. 1 Different situational contexts presented by everyday use of a surface EMG recording device worn on the forearm. 2

Supplementary Fig. 2 Change in signal-to-noise ratio (SNR) after a period of prolonged wear. 3

Supplementary Fig. 3 Feature and classification window selection. 5

Supplementary Fig. 4 Channel selection and effect on classification accuracy. 6

Supplementary Fig. 5 Offline dataset classification accuracy comparison with different algorithms. 8

Supplementary Fig. 6 FPGA resource utilization for each submodule of HD algorithm 9

Supplementary Fig. 7 Power consumption of the system during classification. 10

List of Tables

Supplementary Table 1 sEMG recording sessions for offline analysis consisting of various gesture subsets and situational contexts 2

Supplementary Table 2 EMG features tested for feature selection. 4

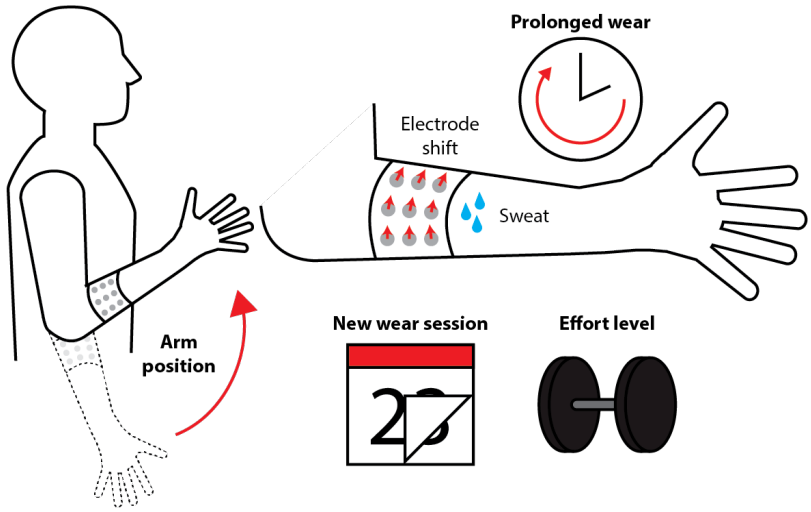
Supplementary Table 3 Inference latency. 9

Supplementary Table 4 Latency for training/update followed by first prediction. 9

Supplementary Table 5 Comparison of this work with other embedded gesture recognition systems. 11

Supplementary Note 1. Offline dataset collection

In order to assess signal quality, validate the HD algorithm, and tune model parameters, we used our system to record an offline dataset of sEMG signals from five healthy, able-bodied, adult, male subjects who were asked to perform gestures in multiple contexts that simulated everyday use of an sEMG acquisition system. Two subjects were experienced with myocontrol and also participated in the in-sensor experiments. The remaining three subjects were new to myocontrol. The changing situational contexts that were included in the offline dataset included changing arm position, changing effort levels (not used for this study), new device wear session, and prolonged wear of the device (Supplementary Figure 1). Recordings were divided into sessions, each representing a different use case or wear condition for the device, and each consisting of a different subset of gestures (Supplementary Table 1).



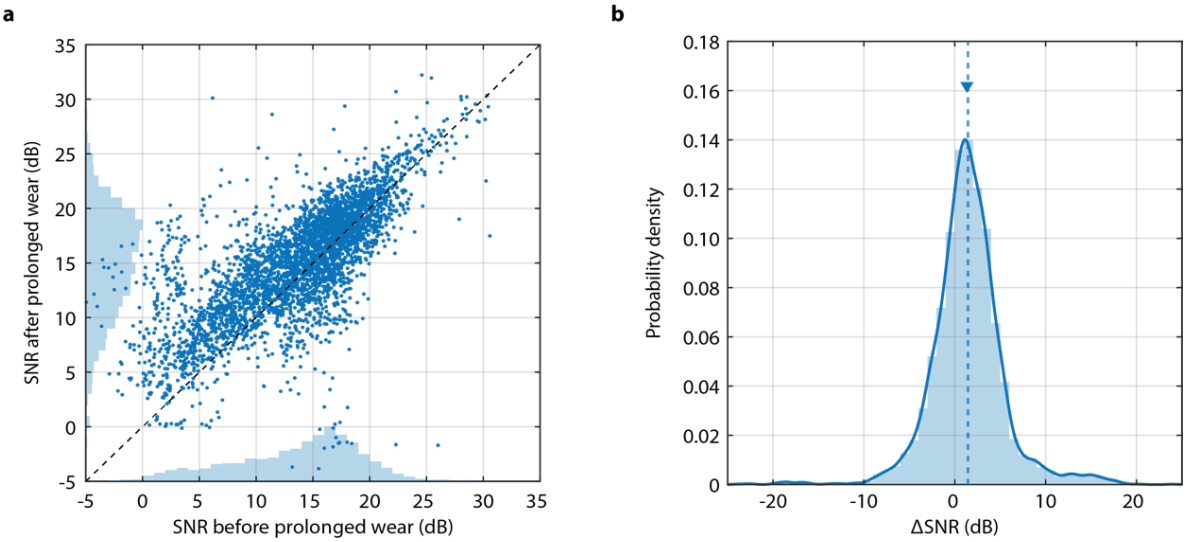
Supplementary Fig. 1 Different situational contexts presented by everyday use of a surface EMG recording device worn on the forearm. Context changes include different arm positions, prolonged wear of the device in a single wear session, differences over multiple wear sessions, and the effort with which gestures are performed. These changes can introduce EMG signal variations which confound the classification of gestures.

Supplementary Table 1 sEMG recording sessions for offline analysis consisting of various gesture subsets and situational contexts.

Session	Gestures	Context
1	Single-DOF	Baseline, relaxed arm position
2	Multi-DOF	Baseline, relaxed arm position
3	Single-DOF	New arm position (arm wrestle)
4	Effort level	Low effort, relaxed arm position
5	Effort level	Medium effort, relaxed arm position
6	Effort level	High effort, relaxed arm position
7	Single-DOF	New wear session, relaxed arm position
8	Single-DOF	Prolonged wear, relaxed arm position

During each recording session, the subject performed 5 trials of each of the gestures. Each trial lasted 11 seconds, with the subject being instructed to begin the gesture during the first two seconds and relax during the last two seconds. The middle 4 second period contained the steady-state portion of the gesture performance, which was the only portion used in this analysis. Trials were each separated by 3 second relax periods.

We used sessions 7 and 8, which were 2 hours apart, to analyze how signal-to-noise ratio of the electrode channels changed over periods of prolonged wear of the device (Supplementary Fig. 2). On average, there was a slight increase of about 1.4 dB in SNR after the 2 hour wear session.



Supplementary Fig. 2 Change in signal-to-noise ratio (SNR) after a period of prolonged wear. **a** Distribution of SNR values upon first donning the device (before prolonged wear) and after 2 hours of continuous wear of the device (after prolonged wear). Univariate histograms of SNR before and after prolonged wear are shown on the axes. Scattered points each represent different combination of subject, channel, and gesture performed. Dashed line shows where SNR before and after prolonged wear would be equivalent. **b** Distribution of the change in SNR from before to after a 2 hour period of prolonged wear. Shaded boxes are the probability density histogram of SNR shift values (ΔSNR), and lines are a Gaussian kernel fit to the distribution. Downward pointing triangle is the median value, and vertical dashed line is the mean value.

Supplementary Note 2. Feature selection and model validation

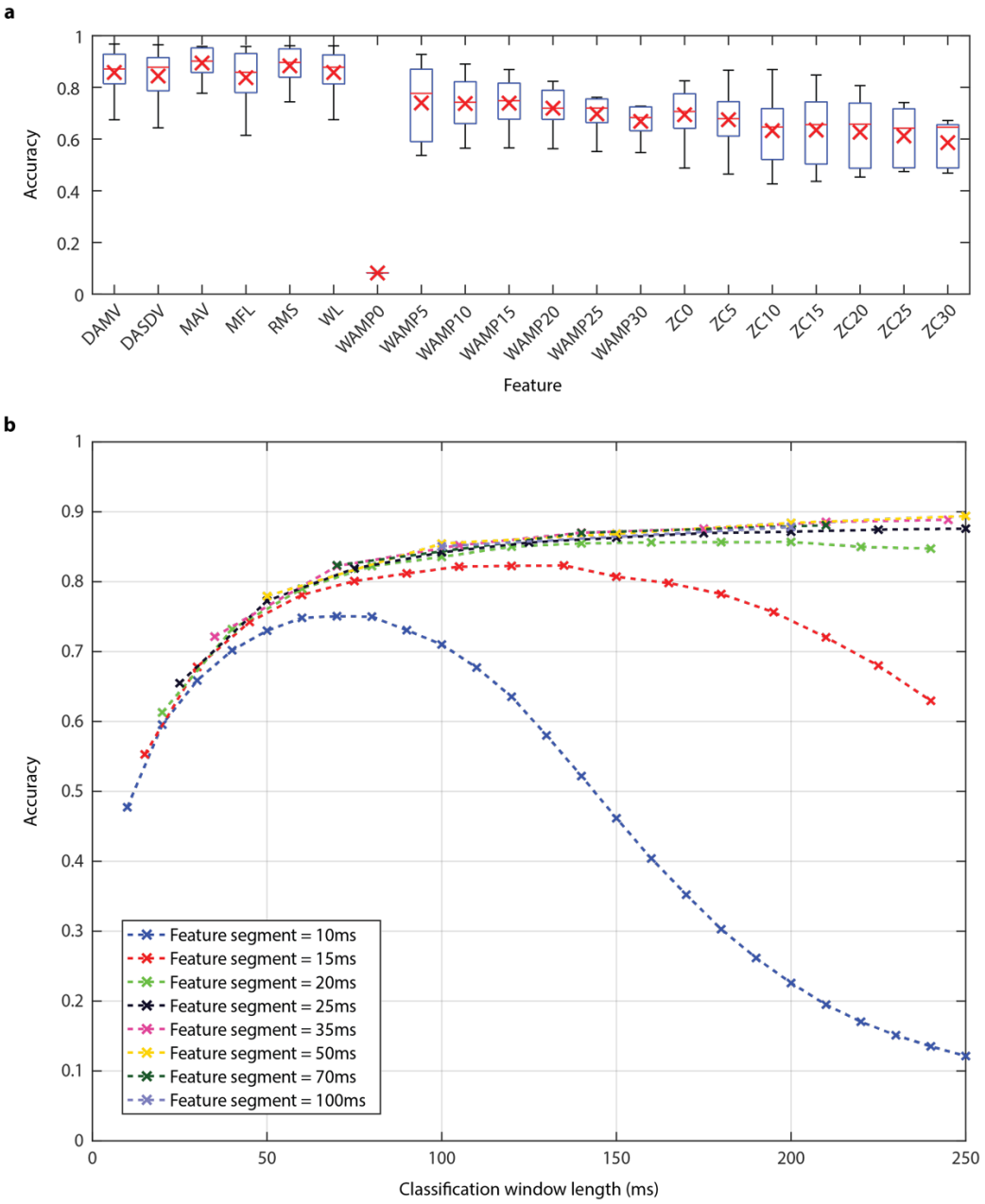
Offline learning and classification experiments were implemented in MATLAB (MathWorks, Inc.). Scripts were run on a single node (24 cores, 64GB RAM) of a research computing cluster. We leveraged MATLAB's Parallel Computing Toolbox to expedite running multiple experiments in parallel.

Cross validation was performed for each experiment by dividing the relevant dataset into a training set, consisting of one randomly selected trial of each gesture, and a testing set, consisting of the remaining four trials of each gesture. This division was repeated 100 times per experiment. We determined the optimal feature function, classification window length, and number of feature segments per classification window heuristically for the HD encoding architecture described in Figure 3a and b. We calculated 8 different features with feature segment lengths of 10, 15, 20, 25, 35, 50, 70, and 100 ms. The tested feature functions (listed in Supplementary Table 2) were chosen based on literature recommendations[1] as well as estimates of implementation complexity. We then created overlapping classification windows which consisted of N consecutive feature segments (e.g., 250 ms classification windows consisting of 5 50 ms feature segments are shown in Figures 3a and b). We tested all possible values of N for creating classification windows less than or equal to 250 ms in length, based on latency requirements recommended by the literature[2,3].

Supplementary Table 2 EMG features tested for feature selection.

DAMV	Difference absolute mean value	$\frac{1}{N}\sum_{k=1}^{N}\lvert\Delta x_k\rvert$
DASDV	Difference absolute standard deviation value	$\sqrt{\frac{1}{N}\sum_{k=1}^{N}\Delta x_k^2}$
MAV	Mean absolute value	$\frac{1}{N}\sum_{k=1}^{N}\lvert x_k\rvert$
MFL	Maximum fractal length	$\log_{10}(\sqrt{\sum_{k=1}^{N}\Delta x_k^2})$
RMS	Root mean square	$\sqrt{\frac{1}{N}\sum_{k=1}^{N}x_k^2}$
WL	Waveform length	$\sum_{k=1}^{N}\lvert\Delta x_k\rvert$
WAMPm	Willison amplitude	$\sum_{k=1}^{N}\begin{cases}1, & \Delta x_k > m\\ 0, & \Delta x_k \le m'\end{cases}$ $m = 0, 5, 10, 15, 20, 25, 30$
ZCm	Zero crossing	$\sum_{k=1}^{N}\begin{cases}1, & \Delta x_k > m\ ,\ \mathrm{sgn}(x_{k-1}) \neq \mathrm{sgn}(x_k)\\ 0, & otherwise\end{cases}$ $m = 0, 5, 10, 15, 20, 25, 30$

To choose the best feature function, we picked the best performing classification window/feature segment size combination for each subject and feature. Supplementary Figure 3a shows the classification accuracy results for each feature. The mean absolute value (MAV) achieved the best classification accuracy across the five subjects. Supplementary Figure 3b shows classification results for different feature segment and classification window length combinations using the MAV feature. For shorter feature segment lengths, there exists an optimal classification window length that is less than 250 ms. However, longer feature segment lengths still achieve better accuracy, with the best performing combination being 50 ms feature segments with a maximal, 250 ms classification window. This combination was used for all other analyses.

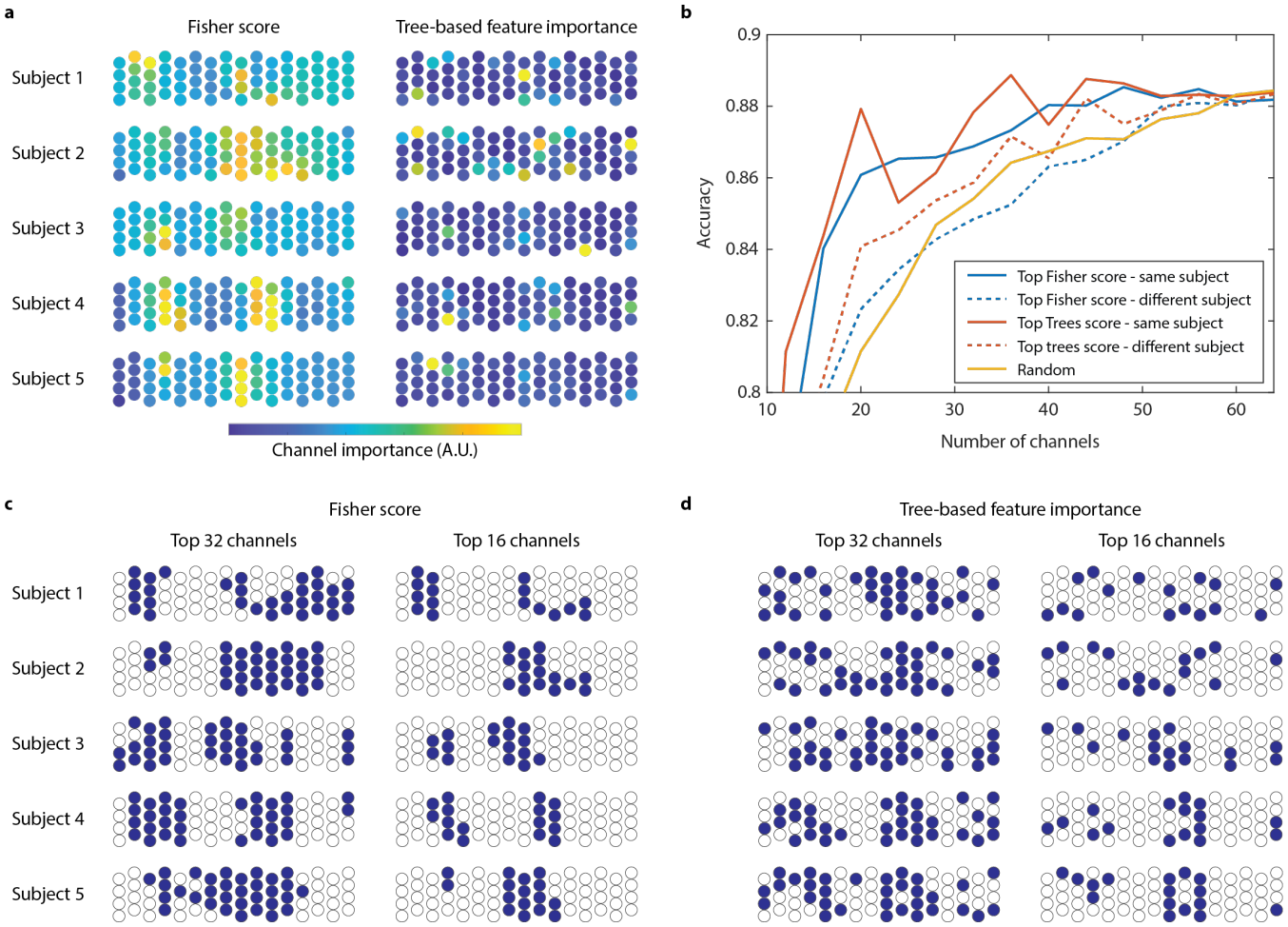


Supplementary Fig. 3 Feature and classification window selection. **a** Comparison of classification accuracy per subject for the different features listed in Supplementary Table 2. For each subject, the best performing feature segment and classification window length were selected for each feature. Boxes cover the middle 50% of values, horizontal red lines denote medians, and whiskers span the full range of data. Red x's denote means. **b** Classification accuracy for different feature segment and classification window lengths for the MAV feature. For each feature segment length, N such feature segments can be combined to form classification windows of up to 250 ms.

To assess whether all 64 electrode channels were necessary for optimal classification performance, we ranked channels based on their importance according to two metrics. The first metric was a univariate Fisher score, which normalizes between-class feature variance by within-class variance[4]. The second metric was a feature gain parameter measured after training a gradient boosted tree-based algorithm for discriminating gestures (500 trees with tree depth of 2). This feature gain represented the relative weight of a feature in making a prediction with the forest. Channel Fisher scores followed similar anatomical patterns, indicating that channels placed over the flexor and extensor muscles produced features that varied more with the different gesture classes (Supplementary Fig. 4a). Boosted trees feature

importances were sparser, with a few channels much more important in making a prediction than the rest.

We ranked the electrodes based on their feature importances for each subject and each metric and used these rankings to gradually drop out channels while applying our hyperdimensional computing algorithm. Classification accuracy gradually decreased as fewer channels were used and steeply degraded when fewer than 16 channels were kept (Supplementary Fig. 4b). Notably, classification accuracy was better preserved when dropping channels based on the subject's own feature importance ranking as opposed to dropping channels based on a different subject's rankings. In the latter case, performance was marginally, if at all, better than randomly dropping channels. Supplementary Figures 4c and 4d show the top 32 and top 16 channel subsets for each subject based on the different rankings. Although there is some overlap, the differences in these subsets are enough to cause the accuracy degradation of a few percent when using channel rankings from a different subject.



Supplementary Fig. 4 Channel selection and effect on classification accuracy. **a** Channel importance as assessed using univariate Fisher score (left) and gain-based feature importance from a gradient boosted tree-based classifier (right) for five subjects. Each circle represents an electrode positioned as such in the array, and the color of the circle represents the normalized feature score for that channel. **b** Classification accuracy degrades when reducing the number of channels for hypervector encoding. Five different methods were used to select the top subset of channels: Fisher scores from the same subject (solid blue line, average over 5 subjects); Fisher scores from a different subject (dotted blue line, average over 5 subjects, each tested with feature scores from all 4 other subjects); tree-based feature importance from the same subject (solid red line); tree-based feature importance from a different subject (dotted red line); and randomly (solid yellow line, averaged over 10 repetitions per subject). **c, d** Top 32 and 16 channel subsets for each subject based on **c** Fisher score and **d** tree-based feature importance.

Supplementary Figure 5 shows the classification accuracy results using the offline dataset. The left portion of Supplementary Figure 5a shows classification accuracy using the hyperdimensional computing model in the baseline situational context for the single- and multi-DOF gesture subsets separately, as well as for all 21 gestures together. The updated model for classifying all 21 gestures was created by concatenating the associative memories previously trained separately for the single- and multi-DOF gesture subsets, requiring no new training computations (Fig. 4a).

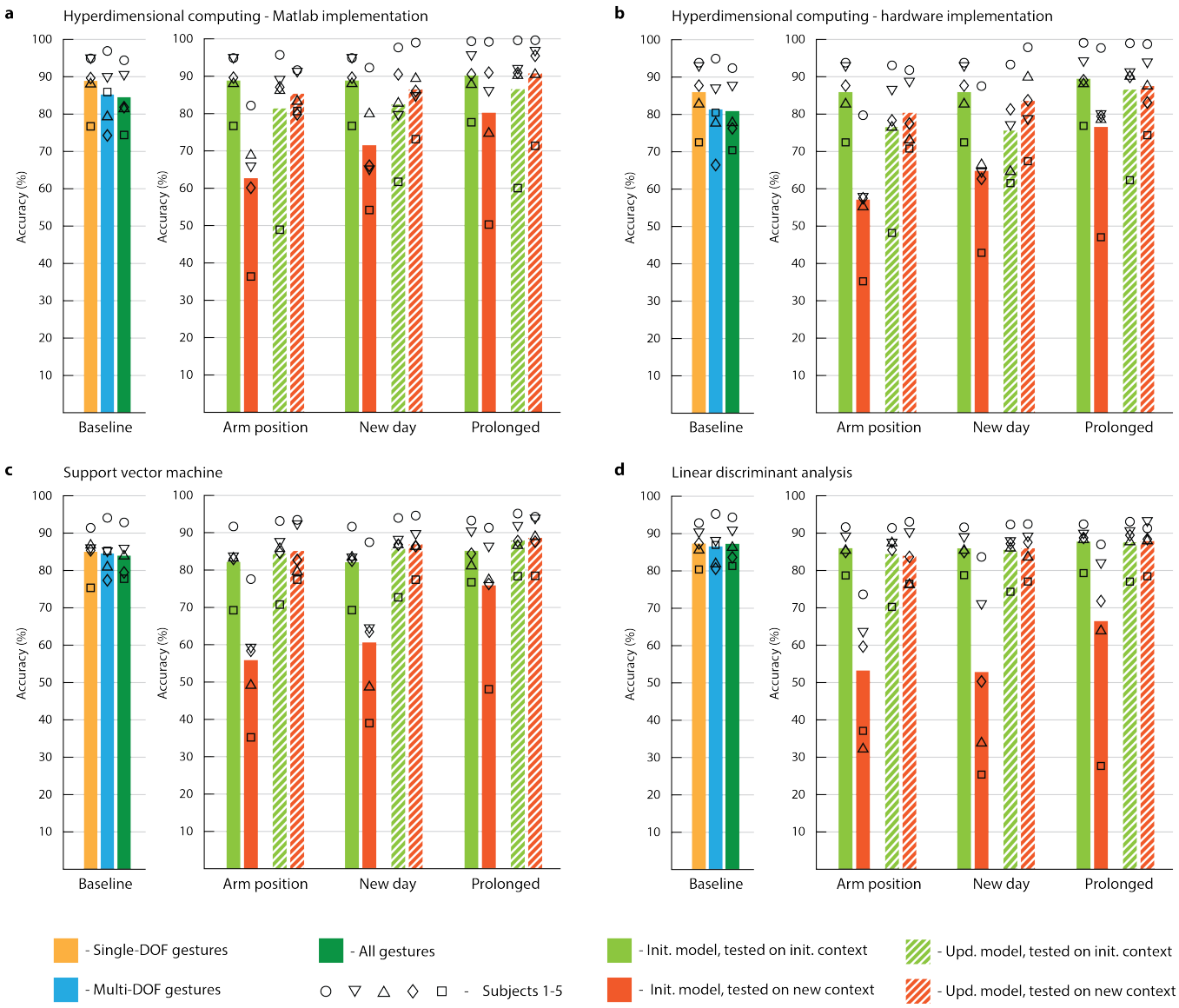
To model fast, online learning, all classification accuracies were measured using cross-validation with one trial of each gesture for training and the remaining trials for testing. We also implemented the standard leave-one-out cross validation (LOOCV) using 4 trials for training and 1 trial for inference. Using LOOCV, the HD classifier achieves 92.47% accuracy for single-DOF, 91.71% for multi-DOF, and 89.85% for all gestures, as compared to the 88.87%, 85.27%, and 84.53% respectively when training with only 1 trial. Thus, although slightly better performance could be achieved with more training data, one-shot learning still produced viable results.

To measure classification accuracy of the HD model across different situational contexts, we performed three experiments in which a model was trained using data from the initial, baseline context and then used to classify gestures in a new dataset from a different context. Supplementary Fig. 5a (right) shows the classification accuracy for the three tested contextual variations: arm position, new wear session, and prolonged wear. Although training and inference were performed on separate datasets, we still used a single trial of each gesture to train the model, and reported accuracies are averages over each of the five training trials. For each of the contextual variations, we saw accuracy degradations of 26.17% (arm position), 17.34% (new day/wear session), and 9.95% (prolonged wear), suggesting that updates were necessary.

To test model updates, prototype hypervectors from the new contexts were trained also using only a single trial of each gesture and then merged with the initial model. We wanted both the initial and new contexts to have equal contributions to the updated model, so the merged hypervector randomly took 50% of its elements from the initial prototype hypervector and the other 50% from the new prototype hypervector (Fig. 4b). Reported classification accuracies are the average of all 25 combinations of initial context prototype and new context prototype. After updating initial models for the new arm position, wear session, and prolonged wear contexts, the classification accuracy on gestures in the new context was improved significantly (22.66%, 14.96%, and 10.51%, respectively) at the expense of small degradation (7.46%, 6.37%, and 3.65%, respectively) in classifying the initial context gestures. This accuracy degradation is due to the bitwise merge operation being only an approximation of the accumulation required to fully retrain a model on data from both contexts. This approximation introduces more error with increasingly distant context prototypes.

Supplementary Fig. 5b shows the performance of the hardware-optimized model (simulated in MATLAB) with the same experiments, demonstrating only a few percent of accuracy degradation when scaling down to 1,000-dimensional hypervectors and using integer features. We also performed the experiments with two traditional machine learning algorithms commonly used for sEMG-based gesture recognition: support vector machine (Supplementary Fig. 5c) and linear discriminant analysis (Supplementary Fig. 5d). Both models were implemented in MATLAB. For the support vector machine model, we used a linear kernel with cost parameter C = 500. For linear discriminant analysis, we implemented the standard MATLAB function included in the Statistics and Machine Learning toolbox, with maximum regularization (gamma = 1) for the covariance matrix estimation.

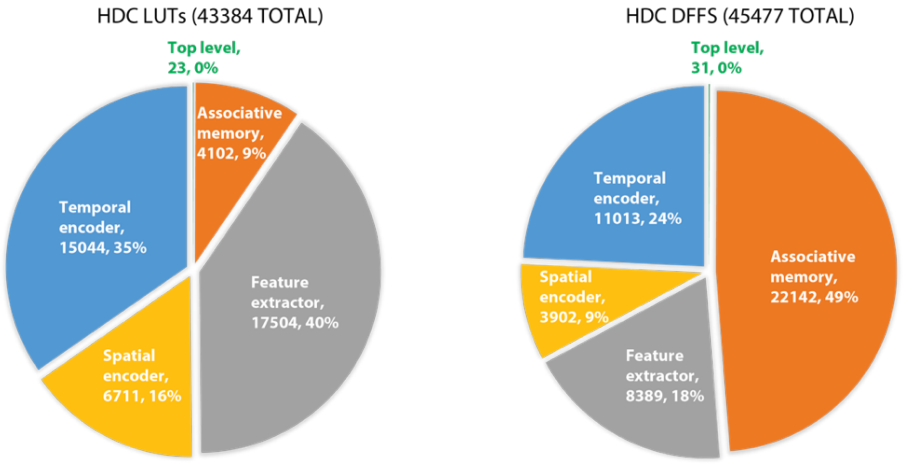
Both of these traditional methods performed similarly well to our model, but with the distinction that the "updated" models in both cases were actually retrained from scratch in order to incorporate training examples from both contexts. This makes them less suitable for online implementation.



Supplementary Fig. 5 Offline dataset classification accuracy comparison with different algorithms. Mean accuracies were calculated one trial of each gesture for training and the remaining four trials for testing, with different gesture subsets (single-DOF gestures only, multi-DOF gestures only, and all gestures combined) within a single situational context (left side of each subfigure), as well as with single-DOF gestures only across different situational contexts (right side of each subfigure). For three different context changes (arm position, new wear session, and prolonged wear), an initial model was trained on the initial context and tested on both the initial context and the new context (left pair of bars). The model was then updated using a single trial of each gesture in the new context, and again tested on both contexts (right pair of bars). Bars represent the mean accuracy across five subjects, with overlaid data points for each individual subject. **a** Results with 10,000-bit hypervector dimension and floating-point features. **b** Results hardware-optimized implementation with 1,000-bit hypervector dimension and 6-bit integer features. **c** Results using a support vector machine model with a linear kernel. **d** Results using linear discriminant analysis.

Supplementary Note 3. FPGA resource utilization

FPGA resource utilization report for the HD algorithm blocks was generated using Synopsys Synplify Pro for Microsemi and illustrated in Supplementary Fig. 6.



Supplementary Fig. 6 FPGA resource utilization for each submodule of HD algorithm based on the number of look-up tables (LUTs, left) and D Flip-Flops (DFFs, right).

Supplementary Note 4. Algorithm latency

The HD algorithm clock cycles was counted through Verilog simulations in Modelsim (Mentor Graphics) and latency was calculated based on that (Supplementary Tables 3 and 4).

Supplementary Table 3 Inference latency. Number of clock cycles for each block is calculated based on HDL simulation results.

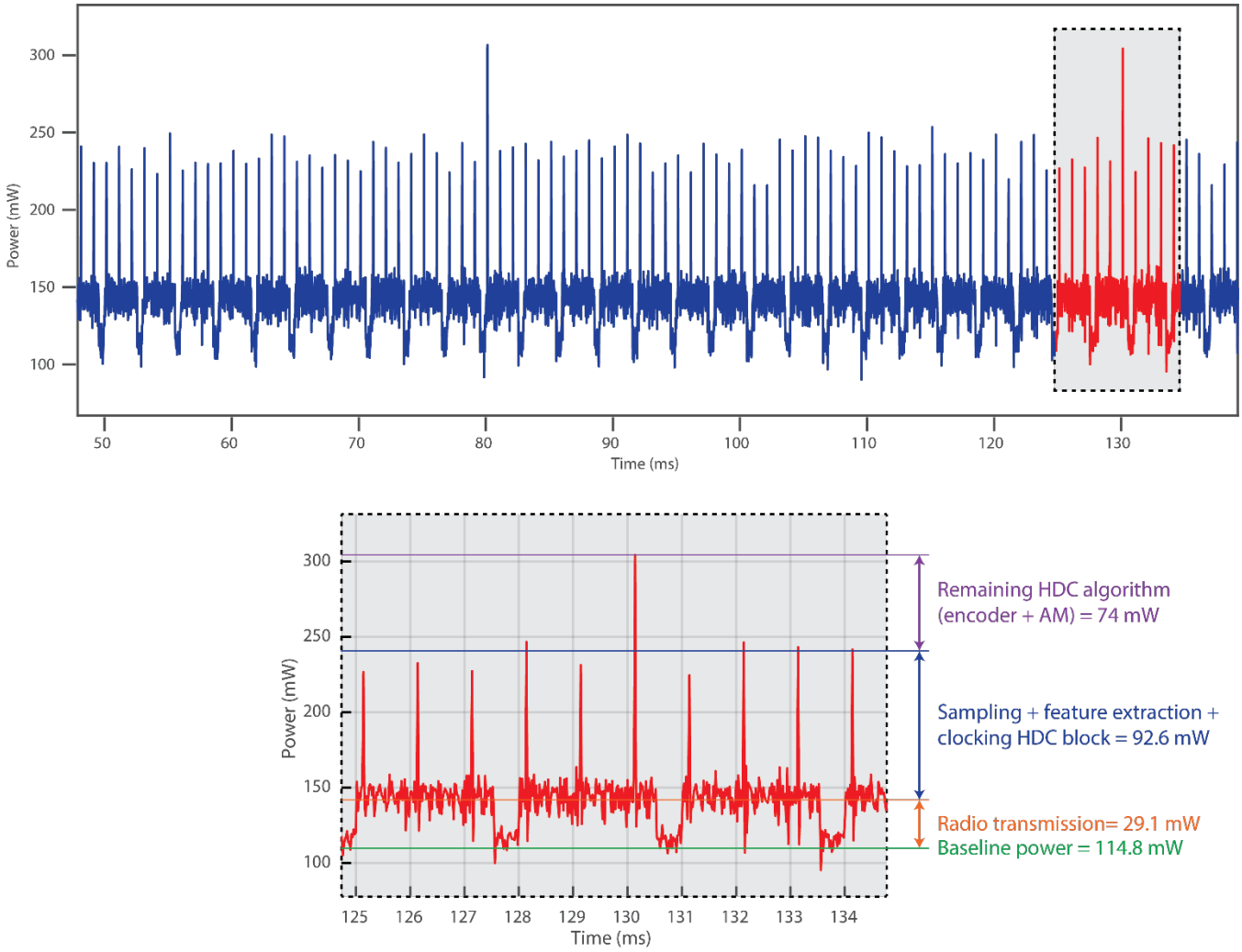
	Clock cycles	Time (w/ 20.48 MHz clock) [µs]
Feature extractor	2	0.10
Spatial encoder	513	25.05
Temporal encoder	1	0.05
Associative memory	23	1.12
Total (predict)	**539**	**26.32**

Supplementary Table 4 Latency for training/update followed by first prediction. Number of clock cycles for each block is calculated based on HDL simulation results.

	Clock cycles	Time (w/ 20.48 MHz clock) [µs]
Feature extractor	2	0.10
Spatial encoder	513	25.05
Temporal encoder	1	0.05
Total (accumulate)	**516**	**25.20**
Feature extractor	2	0.10
Spatial encoder	513	25.05
Temporal encoder	1	0.05
Associative memory (write) + Temporal encoder	1	0.05
Associative memory (predict)	23	1.12
Total (train/update)	**540**	**26.37**

Supplementary Note 5. Power breakdown

The power consumption of the system was measured by an Agilent B2902A precision source/measure unit in 20 µs intervals (Supplementary Fig. 7).



Supplementary Fig. 7 Power consumption of the system during classification. The shorter peaks happening every 1 ms correspond to the accumulation of each new sample for feature extraction. The taller peaks happening every 50 ms include the additional power for the HD encoder and accessing the AM. The radio goes to sleep mode after every 3 samples and wakes up at the arrival of the next sample.

Supplementary Table 5 Comparison of this work with other embedded gesture recognition systems.

		Amma 2015[5]	Benatti 2015[6]	Liu 2017[7]	Pancholi 2019[8]	Benatti 2019[9]	Cerone 2019[10]	This work
System	**Dimensions**	-	85 x 50 mm^2	34 x 25 mm^2	-	-	30 x 25 mm^2 PCB / 3.4 x 3 x 1.5 cm^3 full sensor unit	40 x 33 mm^2 device / 293 x 82 mm^2 electrode array
	Weight	-	-	15.2 g, 4.6 g battery	17.5 g	-	16.7 g	6 g board, 4 g battery, 16 g array & adapter board
	System power	-	36.4 mW streaming / 86mW online recognition	100.6 mW	122.4 mW	10.4 mW	392.7 mW	141.2 mW
EMG acquisition	**Electrode type**	Flexible PCB	Individual Ag-AgCl	Individual Surface Patches	Ag-AgCl Electrode Band	Individual Ag-AgCl	Flexible PCB	Flexible Screen-Printed
	Number of channels	96, differential	8, differential	4, differential	8, differential	8, differential	32, single-ended	64, single-ended
	ADC resolution	-	24 bits	12 bits	24 bits	24 bits	16 bits	15 bits
	Input range	-	±300 mV	-	-	-	10 mVpp	100 mVpp
	Sampling rate	2.048 kHz	1 kHz	1.6 kHz	1 kHz	1 kHz	2.048 kHz	1 kHz
Processing and ML algorithm	**Feature set**	Root mean square	Envelope	Mean absolute value	Root mean square, waveform length, zero crossing, slope sign change	Root mean square	-	Mean absolute value
	Algorithm	Naive Bayes	Floating-point support vector machine	Artificial neural network (ANN)	LDA	Hyperdimensional computing	-	Hyperdimensional computing
	Latency (data window / algorithm)	-	3ms / 1ms	200–250 ms / < 0.2 ms	200 ms with 75 ms increments / -	60 ms / 36 us	-	250 ms / 26 us
	Energy per feature extraction	-	-	-	-	0.86 µJ	-	121.85 µJ
	Energy per classification	-	89.1 µJ	13.8 µJ	-	80.57 µJ	-	1.95 µJ
	Onboard inference	No	Yes	Yes	Yes	Yes	-	Yes
	Onboard training	No	No	No	Yes	Yes	-	Yes
	Onboard updating	No	No	No	No	No	-	Yes
	Dataset	5 healthy	4 healthy	4 healthy	3 healthy + 1 amputee	10 healthy	-	5 healthy (offline) / 2 healthy (online)
	Number of gestures	26 + rest	6 + rest	10	5 + rest	10 + rest	-	20 + rest
	Classification accuracy	90.4%	89.20%	94%	94.14%	85%	-	84.53% (offline) / 92.87% (online)
Wireless	**Protocol**	Wired	Bluetooth	Custom 2.4 GHz	Wired	Bluetooth	WiFi	Custom 2.4 GHz
	EMG streaming	No	Yes	No	No	Yes	Yes	Yes
	Classification streaming	No	Yes	Yes	No	Yes	No	Yes